

Excellent. We'll go step by step. Since you're learning from scratch and want to understand **how things work internally**, we'll start with **Step 1: Word2Vec** instead of immediately using an API.

Step 1 – Word2Vec

What is Word2Vec?

Word2Vec is a Machine Learning algorithm developed by Google in **2013**.

Its purpose is:

Convert words into vectors (numbers) while preserving their meaning.

Instead of storing a word as text:

cat



Word2Vec converts it into something like:

[0.21, -0.54, 0.82, 0.13, ...]



Usually:

- 100 dimensions
- 200 dimensions
- 300 dimensions

Modern embedding models often use **384**, **768**, or **1536** dimensions.

Why Was Word2Vec Invented?

Traditional NLP treats words as completely different.

For example:

Word	Meaning
Car	Vehicle
Automobile	Vehicle

Bag of Words says:

Car ≠ Automobile



TF-IDF also says:

Car ≠ Automobile



But humans know they mean almost the same thing.

Word2Vec learns this automatically.

Simple Example

Imagine these sentences:

I drive a car.



My automobile is red.

The vehicle is expensive.

The algorithm notices:

car



automobile

vehicle

appear in similar contexts.

Therefore, it learns:

car → [0.11, 0.54, ...]



automobile → [0.10, 0.56, ...]

vehicle → [0.12, 0.53, ...]

These vectors are close together.

The Main Idea

Word2Vec follows a simple principle:

Words used in similar contexts usually have similar meanings.

For example:

The cat drinks milk.



The dog drinks milk.

The surrounding words are similar:

drinks



milk

So Word2Vec learns:

cat $\approx$ dog



How Does Word2Vec Learn?

It reads millions (or billions) of sentences.

Suppose the corpus is:

The cat drinks milk.



The dog drinks milk.

The lion drinks water.

The tiger drinks water.

The car needs fuel.

The truck needs fuel.

After seeing these many times, it learns:

cat



dog

lion

tiger

are related.

And

car
truck



are related.

Context Window

This is the most important concept.

Suppose the sentence is:

The cat drinks milk every morning.



Choose a window size of **2**.

Target word:

cat



Window:

The

cat

drinks



So the model learns:

cat

↓

The

drinks



Another target:

drinks



Context:

cat



milk

The model repeatedly learns relationships like these.

Window Size Example

Sentence:

The quick brown fox jumps over the lazy dog



Target:

fox



Window = 2

quick



brown

FOX

jumps

over

The context words are:

quick



brown

jumps

over

The model learns:

"fox is related to quick, brown, jumps, and over."

Two Training Methods

Word2Vec has two architectures.

1. CBOW (Continuous Bag of Words)

CBOW predicts the missing word from its surrounding words.

Example:

The ____ drinks milk.



Input:

The

drinks

milk



Output:

cat



Diagram:

Context Words



Neural Network



Target Word



2. Skip-Gram

Skip-Gram does the opposite.

Given the target word, predict the surrounding words.

Example:

Input:

cat



Output:

The
drinks
milk



Diagram:

Target Word
|
▼
Neural Network
|
▼
Context Words



Skip-Gram generally performs better on smaller datasets and for rare words.

Which One Is Better?

CBOW

Faster

Good for common words

Large datasets

Predict target word

Skip-Gram

Slower

Good for rare words

Small datasets

Predict context words

What Happens Inside?

Suppose your vocabulary is:

cat
dog
milk
water



car

Each word starts with random numbers.

For example:

cat



↓

[0.12, 0.45]

dog



↓

[-0.31, 0.08]

Initially, these numbers are random.

The neural network repeatedly adjusts them while training.

After many iterations, they might become:

cat



↓

[0.81, 0.22]

dog



↓

[0.80, 0.21]

Notice they are now very similar because the model learned they appear in similar contexts.

The Training Cycle

For each sentence:

Sentence



Choose Target Word



Find Context Words



Neural Network Prediction



Calculate Error



Adjust Word Vectors



Repeat Millions of Times



This repeated adjustment is what makes the vectors meaningful.

Real Example

Suppose the training corpus is:

The king rules the kingdom.

The queen rules the kingdom.

The prince is young.

The princess is young.

A man is strong.

A woman is kind.



Eventually, the model learns relationships such as:

King $\approx$ Queen

Prince $\approx$ Princess

Man $\approx$ Woman



This happens without anyone manually telling it those relationships—it emerges from the data.

Why Don't We Implement Word2Vec From Scratch?

Although it's possible, it isn't practical for most applications because:

- It requires **millions of sentences** to produce good embeddings.
- It needs a **neural network** and optimization algorithms like gradient descent.
- Training can take minutes to days depending on the data and hardware.

In real-world C# projects, developers almost always use **pre-trained embedding models** instead of training Word2Vec themselves.

What You Should Learn Next

Before using any embedding API or model, you should understand **how Word2Vec represents words internally**.

The next lesson will cover:

One-Hot Encoding → Neural Network → Embedding Layer

That is the missing piece that explains **how Word2Vec actually converts a word like "cat" into a meaningful vector**. Once you understand that, the rest of modern embeddings (BERT, Sentence Transformers, OpenAI embeddings, etc.) becomes much easier to grasp.